

Frontier Traversal

递归、栈、队列怎样统一成“待展开边界”

408 · 数据结构 Internal Bridge B01

桥梁接口

Visited State + Frontier → Choose Next → Visit → Expand → Update Frontier.

递归调用栈、显式栈和队列都保存已发现但尚未展开的对象；它们的差异在取出顺序，而不是把 Tree、Graph、Stack、Queue 的本体重新定义一遍。

Owners 与停止边界

- DS02 Owns LIFO/FIFO 端点；DS04 Owns 树节点与遍历访问时机；DS07 Owns 图表示、visited 与覆盖。
- 本桥只 Owns 可复用的 frontier 接口、发现时机和覆盖检查。
- 树无环时不需要一般图的 visited；图不连通时必须由外层扫描补成遍历森林。

目录

1	接口语义	2
2	取出规则决定顺序	2
3	代码级最小调用	2
4	检查、复杂度与边界	2
5	做题控制协议	2

1 接口语义

把 `frontier` 分为三种状态：未发现、已发现未展开、已展开。发现时加入 `frontier` 并立即标记，可保证每个对象最多进入边界一次；取出时才处理其邻接对象。若等到取出才标记，两个前驱可能重复入队，复杂度和输出都会改变。

2 取出规则决定顺序

栈是 LIFO，形成深度优先展开；队列是 FIFO，形成按层展开。递归 DFS 只是由语言调用栈隐式保存同一状态。对有权候选，优先队列也可作为 `frontier`，但此时取出的是当前最小优先级，正确性要转交 DS08 的目标不变量。

树与图的同一镜头

树的先序/中序/后序改变“处理根”的时机；图的 DFS/BFS 改变 `frontier` 的取出顺序。树的无环结构允许省略 `visited`，图则必须显式维护它。

3 代码级最小调用

```
1 frontier.push(start); visited[start] = true;
2 while (!frontier.empty()) {
3     auto u = frontier.take();
4     visit(u);
5     for (auto v : expand(u))
6         if (!visited[v]) { visited[v] = true; frontier.push(v); }
7 }
```

具体栈/队列实现分别见 DS02，Tree/Graph 的展开函数见 DS04/DS07；本段只固定交接顺序。

4 检查、复杂度与边界

每个对象最多发现一次、每条邻接关系最多被检查一次，是邻接表遍历 $O(V + E)$ 的证明骨架。检查空 `frontier`、空树、越界起点、非连通分量和重复边；若题目要求最短权值或最小代价，不把 FIFO 的层数结论外推到加权图。

5 做题控制协议

先标出 `frontier`、`visited` 和 `expand` 的对象；再问取出规则；最后写覆盖终止条件。若序列不唯一，说明邻接顺序；若只覆盖一个分量，明确停止位置；若需要全图覆盖，补外层未访问顶点扫描。

压缩复原

忘记 DFS/BFS 口诀时，复原三问：边界保存什么？下一次取谁？发现是在入边界还是出边界时标记？